arXiv:2508.17452v1 [cs.LG] 24 Aug 2025

ReviBranch: Deep Reinforcement Learning for Branch-and-Bound with Revived Trajectories

Jiabao Dou[†], Jiayi Nie[†], Yihang Cheng[‡], Jinwei Liu[†], Yingrui Ji[|], Canran Xiao[¶], Feixiang Du[§], Jiaping Xiao[*]

[†]Hong Kong Baptist University
[‡]Central China Normal University
[¶]Central South University
[|]University of Chinese Academy of Sciences
[§]Tongling University
[*]Nanyang Technological University

Abstract

The Branch-and-bound (B&B) algorithm is the main solver for Mixed Integer Linear Programs (MILPs), where the selection of branching variable is essential to computational efficiency. However, traditional heuristics for branching often fail to generalize across heterogeneous problem instances, while existing learning-based methods such as imitation learning (IL) suffers from dependence on expert demonstration quality, and reinforcement learning (RL) struggles with limitations in sparse rewards and dynamic state representation challenges. To address these issues, we propose ReviBranch, a novel deep RL framework that constructs revived trajectories by reviving explicit historical correspondences between branching decisions and their corresponding graph states along search-tree paths. During training, ReviBranch enables agents to learn from complete structural evolution and temporal dependencies within the branching process. Additionally, we introduce an importance-weighted reward redistribution mechanism that transforms sparse terminal rewards into dense stepwise feedback, addressing the sparse reward challenge. Extensive experiments on different MILP benchmarks demonstrate that ReviBranch outperforms state-of-the-art RL methods, reducing B&B nodes by 4.0% and LP iterations by 2.2% on large-scale instances. The results highlight the robustness and generalizability of ReviBranch across heterogeneous MILP problem classes.

Introduction

Mixed Integer Linear Programming (MILP) provides a fundamental framework for modeling combinatorial optimization problems across production planning, scheduling, facility location, and network design (Wolsey and Nemhauser 1999). The NP-hard nature of these problems requires sophisticated solution methods, with the branch-and-bound (B&B) algorithm serving as the core algorithmic framework in modern MILP solvers (Kianfar 2010).

The B&B algorithm employs a systematic divide-and-conquer strategy, recursively partitioning the solution space through tree search (Land and Doig 2009). At each node of this search tree, a critical decision must be made: select which fractional integer variable to branch (Wolsey and Nemhauser 1999). Crucially, the choice of branching strategy profoundly impacts the algorithm's performance (Achterberg et al. 2008). Different strategies can lead to vastly different search tree sizes, computational complexity, and ultimately, solution quality and solving time (Patel and Chinneck 2007; Morrison et al. 2016). Consequently, developing effective branching strategies is essential for solving complex real-world problems. However, traditional branching heuristics often prove inefficient due to their reliance on simple local criteria without considering global optimization context (Achterberg, Koch, and Martin 2005).

Recent advances in machine learning have introduced sophisticated approaches to branching. Imitation learning (IL) methods train neural networks to mimic expert decisions (Khalil et al. 2016; Gasse et al. 2019), but rely on expensive expert demonstrations (He, Daumé, and Eisner 2014). Reinforcement learning (RL) approaches overcome this through autonomous exploration (Etheve et al. 2020; Parsonson, Laterre, and Barrett 2023), but face three fundamental challenges: *(1) Long-term dependency modeling*: Early branching choices may only reveal their quality after hundreds of subsequent decisions, creating significant credit assignment difficulties. *(2) Dynamic state representation challenges*: Each branching decision fundamentally alters the entire search tree structure, with variables getting fixed, constraints becoming active/inactive, and graph topology changing dynamically (Khalil et al. 2017). This creates difficulties in defining stable state representations that can capture evolving structural information throughout the branching process, as traditional methods struggle to model these complex state transitions effectively (Bengio, Lodi, and Prouvost 2021). *(3) Sparse reward signals*: RL agents typically receive rewards only upon complete problem resolution, as the first integer feasible solution is often found very deep in the search tree. This sparse feedback severely hinders learning efficiency and credit assignment during training (Parsonson, Laterre, and Barrett 2023; Sutton, Barto et al. 1998).

To address these limitations, we consider leveraging his-

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

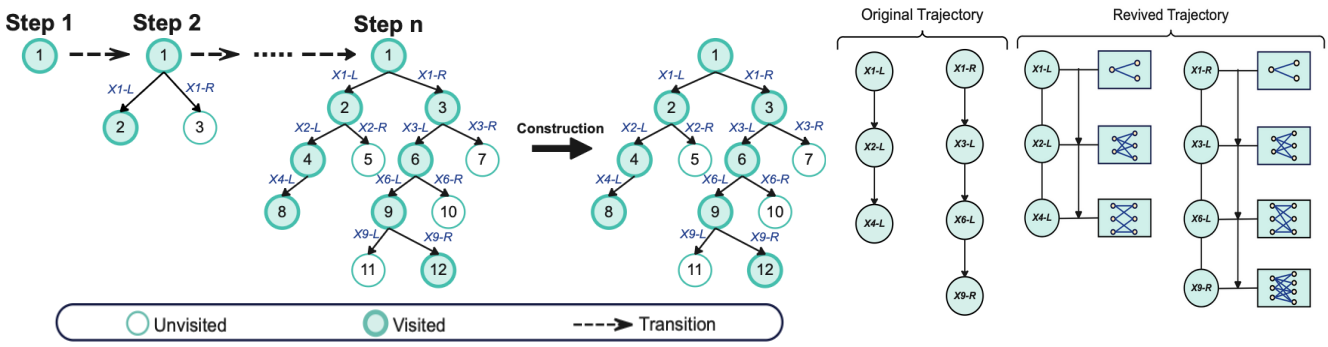


Figure 1: Branch-and-Bound Algorithm and Trajectory Construction. Left: B&B search tree showing branching decisions and state transitions, where each node displays the selected branching variable with constraints $x_i \leq$ (left) and $x_i \geq$ (right). The action represents the choice of branching variable x_i. Right: Comparison between Original Trajectory (action sequences only) and Revived Trajectory (complete state-action correspondences with graph structures).

torical trajectory information to enable agents to learn from past branching decisions. However, during reinforcement learning training, agents inevitably generate suboptimal trajectories through ϵ-greedy exploration strategies (Sutton, Barto et al. 1998). Relying solely on historical action sequences may cause agents to learn erroneous patterns by mistaking local optima for global ones (Schaul et al. 2015). Therefore, we propose ReviBranch, a deep reinforcement learning framework that addresses these challenges through novel architectural and algorithmic innovations.

Specifically, our ReviBranch has three key innovations: **(1)** It employs an Encoder-Revival-Decoder architecture that combines BipartiteGCN (Gasse et al. 2019) for structural graph representation with Graph-Sequence Decoder featuring multidirectional cross-attention mechanisms, enabling agents to capture both spatial problem structure and temporal decision dependencies, addressing Challenge 1. **(2)** It constructs revived trajectories by reviving historical graph states from stored data through distributed storage and reconstructing complete graph state-action correspondences (Figure 1), resolving dynamic state representation challenges and enabling agents to learn from precise temporal context, addressing Challenge 2. **(3)** It develops Importance-Weighted Reward Redistribution (IWRR) that transforms sparse terminal rewards into dense stepwise feedback through temporal importance weighting, enabling effective credit assignment for individual branching decisions and addressing the sparse reward problem, addressing Challenge 3.

Related Work

Classical Branching Heuristics Early approaches include Most Fractional Branching, which selects variables closest to 0.5, and Random Branching for baseline comparison. Pseudocost branching (PB) (Bénichou et al. 1971) uses historical pseudocosts, enabling rapid decisions but suffering from cold starts and poor cross-instance adaptability. Strong branching (SB) (Linderoth and Savelsbergh 1999) overcomes the myopia of PB via costly one-step lookahead, achieving superior solution quality, but at computational costs often orders of magnitude higher than PB.

Learning to Branch To bypass SB's prohibitive online cost, IL methods learn from SB demonstrations. Khalil et al. (Khalil et al. 2016) pioneered this direction using support vector machines to replicate SB rankings, but required costly per-instance training. Gasse et al. (Gasse et al. 2019) introduced bipartite Graph Convolutional Networks that capture variable-constraint relationships through message passing, learning SB policies offline. Zarpellon et al. (Zarpellon et al. 2021) explicitly parameterized search tree states to improve cross-instance generalization. However, these imitation learning approaches remain fundamentally constrained by their expert teachers' performance ceiling. RL offers potential for discovering superior branching strategies through autonomous exploration. Etheve et al. (Etheve et al. 2020) proposed Fixed-depth Monte Carlo Tree Search (FMSTS) to reduce episode length through depth-first trajectory construction. Parsonson et al. (Parsonson, Laterre, and Barrett 2023) developed retro branching by decomposing search trees into subtree trajectories, reducing partial observability but maintaining sparse reward challenges. However, current approaches lack explicit temporal dependency modeling for sequential branching decisions and fail to establish accurate mappings between decisions and evolving states.

Background

Mixed Integer Linear Programming MILP is an optimization problem in which values are assigned to decision variables, subject to linear constraints, so as to minimize a linear objective function. MILPs can be formulated in the following standard form (Wolsey and Nemhauser 1999):

$$\min_x c^T x \quad \text{s.t.} \quad Ax \leq b, \quad l \leq x \leq u, \quad x_j \in \mathbb{Z} \text{ for } j \in \mathcal{I} \tag{1}$$

where $c \in \mathbb{R}^{n_0}$ is the objective coefficient vector, $A \in \mathbb{R}^{m \times n_0}$ is the constraint coefficient matrix, $b \in \mathbb{R}^m$ is the constraint right-hand side vector, $l, u \in \mathbb{R}^{n_0}$ are the lower and upper variable bound vectors, and $\mathcal{I} \subseteq \{1, 2, \ldots, n_0\}$ denotes the set of indices for integer-constrained variables.

To solve MILPs exactly, the B&B algorithm is widely used (Land and Doig 2009).

Branch-and-Bound Algorithm B&B algorithm systematically explores the solution space through recursive partitioning (Land and Doig 2009). At each node, the algorithm solves the LP relaxation, selects a fractional integer variable x_j from $\mathcal{F}(x^*) = \{i \in \mathcal{I} : x_i^* \notin \mathbb{Z}\}$, and creates two child nodes with constraints $x_j \leq \lfloor x_j^* \rfloor$ and $x_j \geq \lceil x_j^* \rceil$, where x_j^* is the fractional value in the current LP solution. SB (Alvarez, Louveaux, and Wehenkel 2017) evaluates each candidate by solving LP relaxations for both potential child nodes, while PB (Bénichou et al. 1971) uses historical statistics to estimate branching effects efficiently, avoiding SB's computational overhead. This motivates machine learning approaches for intelligent variable selection.

Graph Representation for MILP Following Gasse et al. (Gasse et al. 2019), MILP instances are represented as bipartite graphs with constraint and variable vertices, connected when $A_{ij} \neq 0$. Constraint nodes encode right-hand side values and slack information, while variable nodes capture bounds, objective coefficients, and LP solution values. BipartiteGCN processes such graphs through message passing to learn structural representations.

RL Framework Sequential decision-making problems can be formulated as Markov Decision Processes (MDPs) defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where $\mathcal{S}$ represents the state space, $\mathcal{A}$ the action space, $\mathcal{P}$ the transition probabilities, $\mathcal{R}$ the reward function, and γ the discount factor. Q-learning (Watkins and Dayan 1992) aims to learn the optimal action-value function $Q^*(s, a)$ that represents the expected cumulative reward for taking action a in state s and following the optimal policy thereafter. Deep Q-Networks (DQN) (Mnih et al. 2015) address this limitation by approximating the optimal action-value function using neural networks $Q(s, a; \theta) \approx Q^*(s, a)$, where θ represents the network parameters. DQN utilizes experience replay and target networks for training stability.

Method

We present ReviBranch, a deep reinforcement learning framework that employs an Encoder-Revival-Decoder neural network architecture trained via DQN. Our framework introduces three key innovations: (1) Revived Trajectories Construction, which revives historical graph states to construct enhanced trajectories with complete state-action correspondences; (2) Graph-Sequence Decoder, which combines structural graph features with temporal decision sequences through multi-directional attention mechanisms; and (3) IWRR, which transforms sparse terminal rewards into dense stepwise feedback through temporal importance weighting.

Problem Formulation

We formulate the branching variable selection problem as an episodic MDP, where each episode corresponds to solving one MILP instance through a sequence of branching decisions until optimality is reached.

States. At each branching step i, the state s_i is represented as a bipartite graph $G = (C \cup V, E)$, where C and V represent constraint and variable nodes with 5- and 19-dimensional features respectively, and E captures relationships when $A_{ij} \neq 0$. The complete features are provided in Appendix B. The state includes the current action set $\mathcal{A}(s_i)$ of fractional integer variables eligible for branching.

Actions. The action space $\mathcal{A}(s_i) = \{j \in \mathcal{I} : x_j^* \notin \mathbb{Z}\}$ consists of fractional integer variables eligible for branching, where x_j^* is the LP relaxation solution value. Our DQN agent employs a policy $\pi(a_i|s_i)$ to select a variable index $a_i \in \mathcal{A}(s_i)$ for branching.

Transitions. The transition function $\mathcal{P}(s_{i+1}|s_i, a_i)$ is deterministic and follows the standard branch-and-bound mechanism. When variable j is selected for branching at state s_i, the environment creates two child nodes with additional constraints $x_j \leq \lfloor x_j^* \rfloor$ and $x_j \geq \lceil x_j^* \rceil$, where x_j^* is the fractional LP solution value.

Revived Trajectories. As illustrated in Figure 1, we construct revived trajectories $\tau^{\text{revived}} = \{(s_0^{\text{hist}}, a_0), (s_1^{\text{hist}}, a_1), \ldots, (s_{L-1}^{\text{hist}}, a_{L-1})\}$ (where L denotes the trajectory length) by reviving historical graph states s_i^{hist} from stored data and pairing them with corresponding actions a_i (where a_0 denotes the first branching decision, a_1 the second, and so forth). This enables the reconstruction of complete state-action correspondences that capture both structural evolution and temporal dependencies within the search tree.

Reward. We introduce an importance-weighted reward redistribution mechanism. The base reward assigns $r = -1$ at each branching step and $r = 0$ when the action leads to subtree pruning, which is then redistributed using temporal importance weights to provide dense learning signals throughout the episode.

ReviBranch Architecture

Our ReviBranch framework employs a novel Encoder-Revival-Decoder architecture that processes current MILP state and historical context through three main components (Figure 2):

Dual-Feature Encoder The encoder consists of two parallel streams processing different input modalities. The first stream employs BipartiteGCN to extract structural features from the bipartite graph, transforming the discrete optimization structure into continuous variable embeddings $\mathbf{V} \in \mathbb{R}^{n_i \times d}$ that serve as memory context for the Transformer decoder. The second stream processes historical action sequences $\mathbf{a}_{0:L-1} = [a_0, a_1, \ldots, a_{L-1}]$ of variable lengths through a learned embedding mechanism. Each action index a_i representing a selected branching variable is mapped to a d-dimensional embedding vector through learned action embeddings. Positional encodings are added to preserve temporal order information in the sequence representations. The encoded sequence undergoes layer normalization to produce preliminary trajectory representations $\mathbf{R}_{action} \in \mathbb{R}^{L \times d}$ (action representation matrix) containing action embeddings with positional information, which will be used in the revival process to construct complete step-level representations.

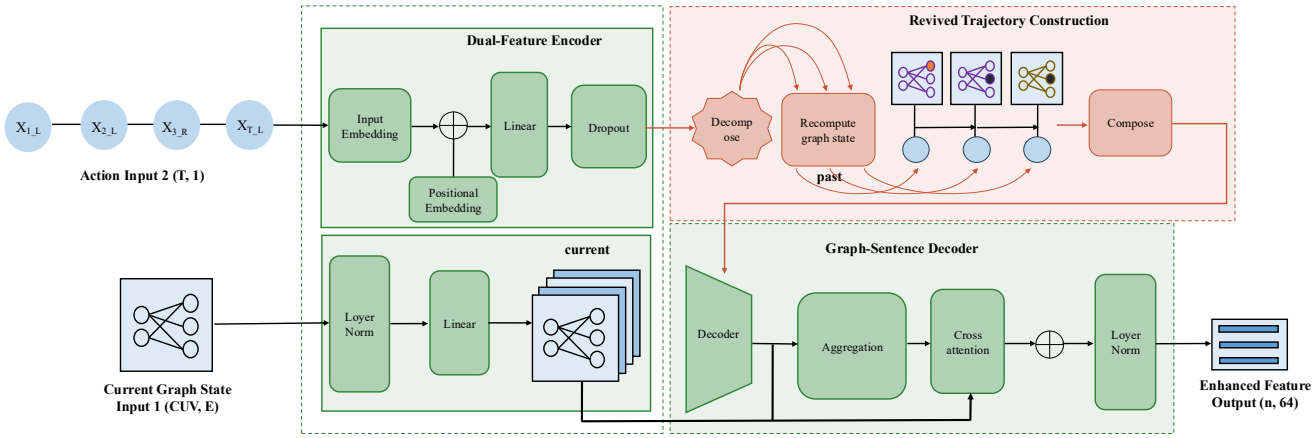


Figure 2: Architecture of ReviBranch framework.

Algorithm 1 Revived Trajectories Construction

Input: Sampled trajectory $\tau=(s_0, a_0, s_1, a_1, \ldots, s_{L-1})$ from replay buffer
Output: Complete revived trajectory $\mathbf{R}_{\text{traj}}=[\mathbf{R}_0, \mathbf{R}_1, \ldots, \mathbf{R}_{L-1}]\in\mathbb{R}^{L\times d}$

for $i=0$ **to** $L-1$ **do**
  $G_i \leftarrow \text{RetrieveStoredState}(s_i)$ {Get stored graph}
  $\mathbf{V}_i \leftarrow \text{BipartiteGCN}(G_i)$ {Reconstruct graph embeddings}
  $\mathbf{a}_i^{\text{emb}} \leftarrow \text{ActionEmb}(a_i)+\text{PE}(i)$ {Generate action embeddings}
  // Step-Level Representation Construction
  $\mathbf{v}_{\text{mean}} \leftarrow \frac{1}{n_i}\sum_{j=1}^{n_i}\mathbf{V}_i[j]$ {Mean pooling}
  $\mathbf{v}_{\text{max}} \leftarrow \max_{j=1}^{n_i}\mathbf{V}_i[j]$ {Max pooling}
  $\mathbf{w} \leftarrow \text{softmax}(\text{MLP}(\mathbf{V}_i))$ {Attention weights}
  $\mathbf{v}_{\text{att}} \leftarrow \sum_{j=1}^{n_i}\mathbf{w}[j]\cdot\mathbf{V}_i[j]$ {Attention pooling}
  $\mathbf{v}_{\text{combined}} \leftarrow \text{MLP}([\mathbf{v}_{\text{mean}}; \mathbf{v}_{\text{max}}; \mathbf{v}_{\text{att}}])$ {Fuse pooled features}
  $\mathbf{R}_i \leftarrow \text{MLP}([\mathbf{v}_{\text{combined}}; \mathbf{a}_i^{\text{emb}}])$ {Fuse with action}
end for
return $\mathbf{R}_{\text{traj}}=[\mathbf{R}_0, \mathbf{R}_1, \ldots, \mathbf{R}_{L-1}]$

Revived Trajectories Construction In DQN training, the agent stores historical experiences in a replay buffer, where each transition contains (s_i, a_i, r_i, s_{i+1}). A general approach would store complete historical context within each transition, requiring each of the L transitions to store up to L historical states, leading to $\mathcal{O}(L^2)$ storage complexity for a trajectory of length L, which is prohibitive for large-scale MILPs. Our revived trajectories construction reduces storage complexity from $\mathcal{O}(L^2)$ to $\mathcal{O}(L)$ through distributed storage. Each transition stores its own complete graph state data $G_i = (C_i, V_i, E_i)$ and action a_i, enabling distributed reconstruction of complete trajectories during training.

During training, given a sampled trajectory $\tau = (s_0, a_0, s_1, a_1, \ldots, s_{L-1})$ from the replay buffer, the revival module reconstructs complete graph state-action correspondences. For each historical step i, the module retrieves the stored graph state data G_i and reconstructs the original graph embeddings $\mathbf{V}_i = \text{BipartiteGCN}(G_i)$. Simultaneously, action embeddings are generated: $\mathbf{a}_i^{emb} = \text{ActionEmb}(a_i) + \text{PE}(i)$, where ActionEmb() is a learned embedding lookup and PE() denotes positional encoding. Step-level representations are then constructed by fusing graph and action information through StepBuilder, which aggregates variable embeddings from the graph state and combines them with the corresponding action embedding to create a unified step representation (detailed fusion mechanism in Algorithm 1):

$$\mathbf{R}_i = \text{StepBuilder}(\mathbf{V}_i, \mathbf{a}_i^{emb}) \tag{2}$$

This revival process is essential because MILP graph structures evolve dynamically during branch-and-bound traversal, where variables get fixed, constraints become active/inactive, and the graph topology changes with each branching decision. The completed revived trajectories $\tau^{\text{revived}} = \{(s_0^{\text{hist}}, a_0), (s_1^{\text{hist}}, a_1), \ldots, (s_{L-1}^{\text{hist}}, a_{L-1})\}$ capture complete graph state-action correspondences while preserving the full structural evolution of the optimization process.

Graph-Sequence Decoder The decoder employs a Transformer decoder architecture with revived trajectory sequence $\mathbf{R}_{traj} \in \mathbb{R}^{L\times d}$ as target input and variable embeddings $\mathbf{V} \in \mathbb{R}^{n_i\times d}$ as memory context. The decoder processes the revived trajectory sequence and performs cross-attention with the variable memory to capture trajectory-variable interactions.

The decoder output undergoes aggregation to extract a unified trajectory representation $\mathbf{R}_{unified} \in \mathbb{R}^d$ through weighted averaging of the sequence. Multi-directional cross-attention consolidates information through three complementary pathways:

$$\mathbf{E}_1 = \text{Attn}(\mathbf{R}_{unified}, \mathbf{V}, \mathbf{V}) \in \mathbb{R}^d \quad \text{(traj-to-var)} \tag{3}$$

$$\mathbf{E}_2 = \text{Attn}(\mathbf{V}, \mathbf{R}_{unified}, \mathbf{R}_{unified}) \in \mathbb{R}^{n_i\times d} \quad \text{(var-to-traj)} \tag{4}$$

$$\mathbf{E}_3 = \text{Attn}(\mathbf{V}, \mathbf{R}_{traj}, \mathbf{R}_{traj}) \in \mathbb{R}^{n_i\times d} \quad \text{(var-to-seq)} \tag{5}$$

The first pathway enables trajectory-to-variable attention for global context understanding. The second pathway allows variable-to-trajectory attention for trajectory-aware variable characterization. The third pathway captures fine-grained variable-to-sequence interactions for temporal pattern modeling. Enhanced variable embeddings are computed through multi-directional cross-attention fusion:

$$\mathbf{V}_{enhanced} = \mathbf{V} + \alpha \cdot \text{Fusion}(\mathbf{E}_1, \mathbf{E}_2, \mathbf{E}_3) \tag{6}$$

where α is a learned scaling parameter. The fusion function is defined as $\text{Fusion}(\mathbf{E}_1, \mathbf{E}_2, \mathbf{E}_3) = \text{MLP}([\mathbf{E}_1; \mathbf{E}_2; \mathbf{E}_3])$, which concatenates and transforms the multi-directional attention outputs, producing enhanced variable embeddings $\mathbf{V}_{enhanced} \in \mathbb{R}^{n_i \times d}$ for Q-value computation.

IWRR We address sparse rewards by redistributing terminal rewards across all steps.At episode termination, we compute temporal importance weights $w_i = (L - i)/L$ to prioritize early decisions, then redistribute terminal rewards as:

$$r_i^{\text{dense}} = R_{\text{terminal}} \times w_i \tag{7}$$

where $R_{\text{terminal}} = \sum_{j=0}^{L-1} r_j$ is the cumulative episode reward (sum of base step rewards), L is the episode length, and $i \in \{0, 1, \ldots, L-1\}$ is the step index. The redistributed rewards are normalized and mapped to $[-0.9, -0.1]$:

$$r_i^{\text{final}} = -0.9 + 0.8 \times \frac{w_i - \frac{1}{L}}{1 - \frac{1}{L}} \tag{8}$$

This temporal weighting mechanism provides dense learning signals while maintaining training stability.

Training Procedure Our ReviBranch framework employs standard DQN training with experience replay. During each episode, the agent collects transitions (s_i, a_i, r_i, s_{i+1}) while maintaining action histories and storing graph state data. At episode termination, sparse environmental rewards are transformed into dense signals through Importance-Weighted Reward Redistribution and stored in the replay buffer. For network training, mini-batches are sampled from the buffer, revived trajectories are constructed via the revival module, and the network is updated using standard DQN loss with dense rewards:

$$\mathcal{L} = \mathbb{E}[(r_i^{\text{dense}} + \gamma \max_{a'} Q_{\text{target}}(s_{i+1}, a') - Q(s_i, a_i))^2] \tag{9}$$

Complete training algorithm is provided in Appendix A.

Experiments & Results

We conduct comparative experiments evaluating ReviBranch against the IL approach of Gasse et al. (2019), RL methods (the state-of-the-art Retro Branching), and traditional heuristic baselines. We also conduct ablation studies to validate each component. The source code is included in the supplementary material and will be released on GitHub upon paper acceptance.

Experimental Setup

Environment Configuration. We employ SCIP 7.0.0 (Achterberg 2009) as the backend mixed-integer programming solver, interfaced through the Ecole framework (Prouvost et al. 2020). We set a time limit of one hour per instance, disable solver restarts, and restrict cutting plane generation to the root node only. The complete environment configuration is provided in Appendix A.

Benchmark. We evaluate on three NP-hard combinatorial optimization benchmarks with three difficulty levels each: **Set Covering** instances: Easy (200×400), Medium (500×1000), Hard (1000×1500). **Combinatorial Auction** instances: Easy (20×60), Medium (30×80), Hard (50×100). **Capacitated Facility Location** instances: Easy (10×15), Medium (25×25), Hard (40×30).

Training. We train ReviBranch using Deep Q-Network with prioritized experience replay on each benchmark separately, using the medium difficulty level for training. Training instances are dynamically generated during reinforcement learning using Ecole's instance generators with random seeds. Testing uses pre-generated instances with distinct seeds to ensure no data leakage and proper generalization assessment. Training is set to a maximum of 600,000 epochs with early stopping based on validation performance. Detailed training configurations, hyperparameters, hardware specifications and training curve are provided in Appendix A.

Testing. Each method is evaluated on every testing instance using 5 different random seeds. For each benchmark, 50 testing instances per difficulty level (easy/medium/hard) are randomly generated with fixed seeds, totaling 150 instances per problem class, which were unseen during training. We report the geometric mean of LP iterations and branch-and-bound nodes as primary metrics, following standard practice in MILP literature. LP iterations serve as a proxy for computational efficiency, as fewer LP iterations typically correlate with reduced inference time per branching decision. All metrics include average per-instance standard deviation (in percentage).

Baseline Methods. We compare ReviBranch against representative branching strategies spanning traditional heuristics and machine learning approaches. **Machine learning baselines:** Imitation learning (IL) (Gasse et al. 2019) and Retro Branching (Parsonson, Laterre, and Barrett 2023). **Traditional baselines:** Strong Branching (SB) (Alvarez, Louveaux, and Wehenkel 2017), Pseudocost Branching (PB) (Bénichou et al. 1971), and SCIP's default reliability pseudocost branching.

Results & Analysis

Performance Comparison with RL Baselines Table 1 reveals several critical insights about ReviBranch's performance characteristics across different problem and difficulty levels. A key finding is that ReviBranch exhibits a distinctive performance pattern that varies with problem complexity, fundamentally differing from the retro branching method. On Set Covering problems, the performance gap between ReviBranch and Retro increases substantially as complexity

Set Covering						
Method	Easy		Medium		Hard	
	LP↓	Nodes↓	LP↓	Nodes↓	LP↓	Nodes↓
SB	847.3±15.2%	19.6±17.9%	5835±16.9%	119.1±19.4%	17259.5±18.1%	532.4±19.9%
SCIP Default	1046.9±8.7%	24.8±8.1%	7051±11.2%	186.4±13.2%	21675.6±14.4%	746.4±16.3%
PB	1494.6±20.8%	43.1±26.0%	11807±18.2%	315.3±22.8%	28406.8±21.8%	889.1±23.1%
IL	973.5±13.9%	23.2±16.4%	6334±15.1%	161±20.9%	18796.1±18.6%	638.6±19.4%
Retro	**895.7±15.7%**↓	**22.4±13.1%**↓	6307.4±19.6%	159.6±21.8%	19102.5±16.2%	657.2±23.1%
Revi	962.9±17.3%	25.9±19.8%	**6095±12.7%**↓	**151.7±17.3%**↓	**18031.7±19.4%**↓	**583.7±20.8%**↓
Capacitated Facility Location						
Method	Easy		Medium		Hard	
	LP↓	Nodes↓	LP↓	Nodes↓	LP↓	Nodes↓
SB	46.4±11.7%	16.6±12.1%	168.1±14.1%	62.5±13.9%	515.2±13.1%	153.8±10.7%
SCIP Default	53.6±13.6%	19.1±11.3%	185.7±10.0%	64.3±11.8%	624.9±9.6%	184.5±11.0%
PB	50.5±13.6%	17.9±16.1%	177.4±12.7%	67.2±15.8%	577.9±15.9%	180.1±16.8%
IL	45.7±9.3%	16.8±9.9%	**159.8±10.5%**↓	**57.7±12.4%**↓	542.6±12.4%	165.9±10.5%
Retro	**46.3±12.9%**↓	**16.2±11.0%**↓	162.2±9.3%	58.6±10.1%	533.8±11.7%	161.2±10.9%
Revi	49.6±10.3%	17.3±12.7%	171.7±12.7%	61.2±11.3%	**529.7±12.6%**↓	**159.9±11.4%**↓
Combinatorial Auction						
Method	Easy		Medium		Hard	
	LP↓	Nodes↓	LP↓	Nodes↓	LP↓	Nodes↓
SB	34.7±13.1%	11.9±10.3%	115.8±15.6%	32.0±12.1%	526.8±18.9%	197.3±15.7%
SCIP Default	47.8±15.3%	18.7±12.9%	136.5±19.4%	45.1±15.2%	637.9±15.5%	231.0±17.3%
PB	74.1±21.4%	28.1±23.7%	226.5±23.2%	88.9±25.3%	1210.6±20.8%	562.3±24.9%
IL	**40.9±16.1%**↓	**12.1±13.5%**↓	131.9±17.6%	36.8±16.3%	569.8±18.2%	205.3±16.9%
Retro	41.6±18.3%	12.5±15.7%	126.4±15.0%	36.3±14.7%	552.1±13.6%	203.6±13.8%
Revi	45.3± 17.4%	13.8±14.2%	**125.9± 19.7%**↓	**35.7±17.6%**↓	**547.8± 16.5%**↓	**200.9±15.7%**↓

Table 1: Performance comparison of branching methods on three MILP benchmarks across difficulty levels. Results show geometric mean ± standard deviation of LP iterations and B&B nodes over 50 test instances per difficulty level. **Lower values indicate better performance**. Bold indicates best performance among learning-based methods.

grows: from ReviBranch performing marginally worse on Easy instances (962.9 vs 895.7 LP iterations, representing a 7.5% increase, worse performance) to achieving substantial advantages on Hard instances (18031.7 vs 19102.5, achieving a 5.6% improvement, better performance). This progressive performance improvement is even more pronounced in the number of tree nodes, where ReviBranch achieves 11.2% fewer nodes on Hard instances (583.7 vs 657.2). These results indicate that simple instances can be effectively solved through the local optimization strategies employed by retro branching, whereas complex instances require the global temporal reasoning capabilities that ReviBranch provides.

This pattern is validated by trajectory length sensitivity analysis, which shows that longer trajectories improve performance. The model with trajectory length 25 generates fewer nodes and shorter inference time than trajectory length 10, and as trajectory length gradually increases, both nodes and inference time further decrease.

Therefore, ReviBranch's benefits become apparent when branching decisions exhibit strong sequential dependencies, where early branching choices significantly influence the effectiveness of later decisions through constraint propagation and search space reduction. This characteristic intensifies with problem complexity as larger instances create deeper interdependencies between variables. In contrast, simpler instances often allow for more independent branching decisions, where each choice has limited impact on subsequent decisions. In such cases, ReviBranch's temporal modeling becomes less advantageous compared to direct heuristics focusing on immediate local variable properties rather than historical patterns.

Overall, across medium and hard difficulty levels (large-scale instances) on all three benchmarks, ReviBranch achieves an average reduction of 4.0% in branch-and-bound nodes and 2.2% in LP iterations compared to state-of-the-art reinforcement learning methods.

Comparison with other methods ReviBranch consistently outperforms both traditional heuristics and learning-based methods across most benchmarks. While SB achieves the best performance with 4.5% fewer LP iterations than

ReviBranch on Hard Set Covering instances, ReviBranch significantly outperforms PB by 36.5% and IL baselines by 4.1% on the same instances, with similar advantages observed across other benchmarks. Notably, ReviBranch demonstrates superior performance over IL methods on large-scale instances, where the benefits of temporal reasoning become more pronounced.

Generalization analysis Despite training only on medium-scale instances, ReviBranch demonstrates reasonable generalization capabilities across two dimensions. Our method exhibits effective adaptability when problem complexity increases. For Set Covering, ReviBranch shows progressively expanding performance advantages from medium instances (LP: 6095 vs 6307 Retro) to hard instances (LP: 18031.7 vs 19102.5 Retro). For Capacitated Facility Location, it maintains competitive performance across difficulty levels, demonstrating robustness to problem scale variations. Additionally, ReviBranch achieves strong performance across three structurally distinct benchmarks, indicating that ReviBranch learns generalizable branching strategies that transfer across different optimization landscapes.

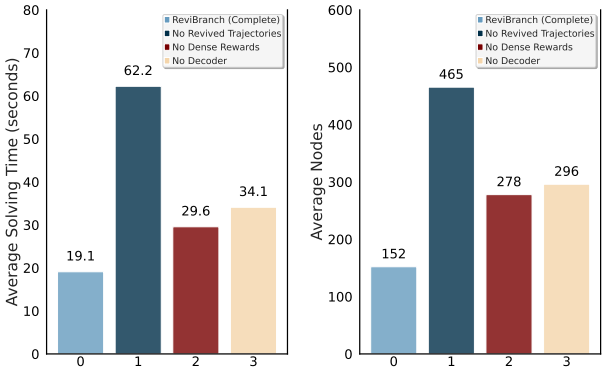


Figure 3: Ablation study results on Set Covering instances. Performance comparison of three ablation variants: (1) No Revived Trajectories, (2) No Dense Rewards, and (3) No Decoder. Results show average solving time and nodes with standard deviation over 50 medium-scale instances across three random seeds.

Ablation Experiments We design three ablation variants: **no revived trajectories**, **no dense rewards**, and **no decoder**. To ensure statistical reliability of our results, we train each variant with three different random seeds and report average performance on 50 medium Set Covering instances. As shown in Figure 3, the no revived trajectories variant exhibits significant performance degradation: solving time increases by 110.1% compared to no dense rewards and 82.5% compared to no decoder; nodes increase by 67.3% compared to no dense rewards and 57.1% compared to no decoder. This occurs because removing revived trajectories prevents tracking graph evolution, leading to poor variable selection and prolonged solving time. Meanwhile, exploration stability deteriorates, with standard deviation increasing by 125% (time) and 79% (nodes) relative to no dense rewards, and 69% (time) and 34% (nodes) relative to no decoder. In contrast, the no dense rewards and no decoder variants show moderate performance decline, indicating that dense rewards and decoder components, while valuable, are non-decisive and primarily provide technical support for effective utilization of revived trajectories.

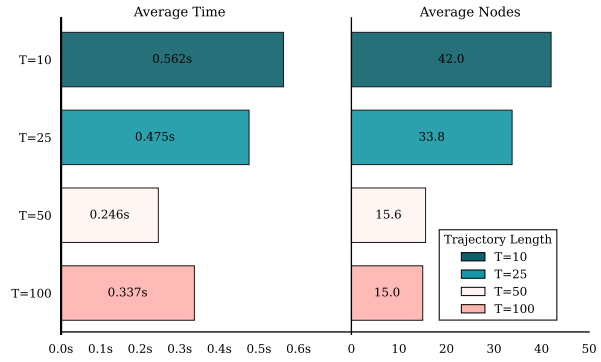


Figure 4: Trajectory length sensitivity analysis on Combinatorial Auction instances (20×60 scale). Results show the trade-off between model performance (nodes) and computational efficiency (inference time) across different trajectory lengths (T=10, 25, 50, 100).

Trajectory length sensitivity analysis We conduct trajectory length sensitivity analysis to understand its impact on ReviBranch performance. As shown in Figure 4, we test four different trajectory length configurations (T=10, 25, 50, 100) on 20×60 scale Combinatorial Auction instances. The experimental results show that the model with trajectory length 10 performs worst, with an average inference time of 0.562 seconds and requiring 42.0 nodes. Model performance further improves as trajectory length increases from 25 to 100.

However, we find that although the trajectory length 100 model has 0.6 fewer nodes than trajectory length 50, the inference time of trajectory length 100 is 0.357 seconds, which is 45% longer than that of trajectory length 50 (0.246 seconds). As the problem complexity increases, this performance pattern requires increasing computational costs. This overhead affects inference time, as each action selection requires recomputing the entire historical trajectory.

Conclusion

We propose ReviBranch, a deep reinforcement learning framework that advances MILP branching through revived trajectory construction. By establishing explicit mappings between branching decisions and evolving graph states, ReviBranch addresses fundamental challenges: sparse rewards, long-term dependencies, and dynamic state representation. Our experiments demonstrate that ReviBranch outperforms state-of-the-art RL methods on large-scale instances, reducing branch-and-bound nodes by 4.0% and LP iterations by 2.2%. The framework exhibits strong generalization across diverse problem classes, with advantages most pronounced on complex instances.

ReviBranch establishes temporal reasoning as an effective paradigm for sequential optimization decisions. Future work may explore adaptive trajectory selection mechanisms and cross-trajectory reward attribution, while extending the approach to other solver components and sequential decision problems in combinatorial optimization.

References

Achterberg, T. 2009. SCIP: solving constraint integer programs. *Mathematical Programming Computation*, 1(1): 1–41.

Achterberg, T.; Berthold, T.; Koch, T.; and Wolter, K. 2008. Constraint integer programming: A new approach to integrate CP and MIP. In *International Conference on Integration of Artificial Intelligence (AI) and Operations Research (OR) Techniques in Constraint Programming*, 6–20. Springer.

Achterberg, T.; Koch, T.; and Martin, A. 2005. Branching rules revisited. *Operations Research Letters*, 33(1): 42–54.

Alvarez, A. M.; Louveaux, Q.; and Wehenkel, L. 2017. A machine learning-based approximation of strong branching. *INFORMS Journal on Computing*, 29(1): 185–195.

Bengio, Y.; Lodi, A.; and Prouvost, A. 2021. Machine learning for combinatorial optimization: a methodological tour d'horizon. *European Journal of Operational Research*, 290(2): 405–421.

Bénichou, M.; Gauthier, J.-M.; Girodet, P.; Hentges, G.; Ribière, G.; and Vincent, O. 1971. Experiments in mixed-integer linear programming. *Mathematical programming*, 1(1): 76–94.

Etheve, M.; Alès, Z.; Bissuel, C.; Juan, O.; and Kedad-Sidhoum, S. 2020. Reinforcement learning for variable selection in a branch and bound algorithm. In *International conference on integration of constraint programming, artificial intelligence, and operations research*, 176–185. Springer.

Gasse, M.; Chételat, D.; Ferroni, N.; Charlin, L.; and Lodi, A. 2019. Exact combinatorial optimization with graph convolutional neural networks. *Advances in neural information processing systems*, 32.

He, H.; Daumé, H.; and Eisner, J. 2014. Learning to search in branch and bound algorithms. *Advances in neural information processing systems*, 27.

Khalil, E.; Le Bodic, P.; Song, L.; Nemhauser, G.; and Dilkina, B. 2016. Learning to branch in mixed integer programming. In *Proceedings of the AAAI conference on artificial intelligence*, volume 30.

Khalil, E. B.; Dilkina, B.; Nemhauser, G. L.; Ahmed, S.; and Shao, Y. 2017. Learning to Run Heuristics in Tree Search. In *Ijcai*, volume 2017, 659–666.

Kianfar, K. 2010. Branch-and-bound algorithms. *Wiley Encyclopedia of Operations Research and Management Science. John Wiley & Sons, Inc*.

Land, A. H.; and Doig, A. G. 2009. An automatic method for solving discrete programming problems. In *50 Years of Integer Programming 1958-2008: From the Early Years to the State-of-the-Art*, 105–132. Springer.

Linderoth, J. T.; and Savelsbergh, M. W. 1999. A computational study of search strategies for mixed integer programming. *INFORMS Journal on Computing*, 11(2): 173–187.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; et al. 2015. Human-level control through deep reinforcement learning. *nature*, 518(7540): 529–533.

Morrison, D. R.; Jacobson, S. H.; Sauppe, J. J.; and Sewell, E. C. 2016. Branch-and-bound algorithms: A survey of recent advances in searching, branching, and pruning. *Discrete Optimization*, 19: 79–102.

Parsonson, C. W.; Laterre, A.; and Barrett, T. D. 2023. Reinforcement learning for branch-and-bound optimisation using retrospective trajectories. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, 4061–4069.

Patel, J.; and Chinneck, J. W. 2007. Active-constraint variable ordering for faster feasibility of mixed integer linear programs. *Mathematical Programming*, 110(3): 445–474.

Prouvost, A.; Dumouchelle, J.; Scavuzzo, L.; Gasse, M.; Chételat, D.; and Lodi, A. 2020. Ecole: A gym-like library for machine learning in combinatorial optimization solvers. *arXiv preprint arXiv:2011.06069*.

Schaul, T.; Quan, J.; Antonoglou, I.; and Silver, D. 2015. Prioritized experience replay. *arXiv preprint arXiv:1511.05952*.

Sutton, R. S.; Barto, A. G.; et al. 1998. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge.

Watkins, C. J.; and Dayan, P. 1992. Q-learning. *Machine learning*, 8(3): 279–292.

Wolsey, L. A.; and Nemhauser, G. L. 1999. *Integer and combinatorial optimization*. John Wiley & Sons.

Zarpellon, G.; Jo, J.; Lodi, A.; and Bengio, Y. 2021. Parameterizing branch-and-bound search trees to learn branching policies. In *Proceedings of the aaai conference on artificial intelligence*, volume 35, 3931–3939.

Reproducibility Checklist

Instructions for Authors:

This document outlines key aspects for assessing reproducibility. Please provide your input by editing this `.tex` file directly.

For each question (that applies), replace the "Type your response here" text with your answer.

Example: If a question appears as

```
\question{Proofs of all novel claims
are included} {(yes/partial/no)}
Type your response here
```

you would change it to:

```
\question{Proofs of all novel claims
are included} {(yes/partial/no)}
yes
```

Please make sure to:

- Replace ONLY the "Type your response here" text and nothing else.
- Use one of the options listed for that question (e.g., **yes**, **no**, **partial**, or **NA**).
- **Not** modify any other part of the `\question` command or any other lines in this document.

You can `\input` this .tex file right before `\end{document}` of your main file or compile it as a stand-alone document. Check the instructions on your conference's website to see if you will be asked to provide this checklist with your paper or separately.

---

1. General Paper Structure

1.1. Includes a conceptual outline and/or pseudocode description of AI methods introduced (yes/partial/no/NA) yes

1.2. Clearly delineates statements that are opinions, hypothesis, and speculation from objective facts and results (yes/no) yes

1.3. Provides well-marked pedagogical references for less-familiar readers to gain background necessary to replicate the paper (yes/no) yes

2. Theoretical Contributions

2.1. Does this paper make theoretical contributions? (yes/no) no

If yes, please address the following points:

2.2. All assumptions and restrictions are stated clearly and formally (yes/partial/no) Type your response here

2.3. All novel claims are stated formally (e.g., in theorem statements) (yes/partial/no) Type your response here

2.4. Proofs of all novel claims are included (yes/partial/no) Type your response here

2.5. Proof sketches or intuitions are given for complex and/or novel results (yes/partial/no) Type your response here

2.6. Appropriate citations to theoretical tools used are given (yes/partial/no) Type your response here

2.7. All theoretical claims are demonstrated empirically to hold (yes/partial/no/NA) Type your response here

2.8. All experimental code used to eliminate or disprove claims is included (yes/no/NA) Type your response here

3. Dataset Usage

3.1. Does this paper rely on one or more datasets? (yes/no) yes

If yes, please address the following points:

3.2. A motivation is given for why the experiments are conducted on the selected datasets (yes/partial/no/NA) yes

3.3. All novel datasets introduced in this paper are included in a data appendix (yes/partial/no/NA) yes

3.4. All novel datasets introduced in this paper will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no/NA) yes

3.5. All datasets drawn from the existing literature (potentially including authors' own previously published work) are accompanied by appropriate citations (yes/no/NA) yes

3.6. All datasets drawn from the existing literature (potentially including authors' own previously published work) are publicly available (yes/partial/no/NA) yes

3.7. All datasets that are not publicly available are described in detail, with explanation why publicly available alternatives are not scientifically satisficing (yes/partial/no/NA) yes

4. Computational Experiments

4.1. Does this paper include computational experiments? (yes/no) yes

If yes, please address the following points:

4.2. This paper states the number and range of values tried per (hyper-) parameter during development of the paper, along with the criterion used for selecting the final parameter setting (yes/partial/no/NA) yes

4.3. Any code required for pre-processing data is included in the appendix (yes/partial/no) yes

4.4. All source code required for conducting and analyzing the experiments is included in a code appendix (yes/partial/no) yes

4.5. All source code required for conducting and analyzing the experiments will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no) yes

4.6. All source code implementing new methods have comments detailing the implementation, with references to the paper where each step comes from (yes/partial/no) yes

4.7. If an algorithm depends on randomness, then the method used for setting seeds is described in a way sufficient to allow replication of results (yes/partial/no/NA) yes

4.8. This paper specifies the computing infrastructure used for running experiments (hardware and software), including GPU/CPU models; amount of memory; operating system; names and versions of relevant software libraries and frameworks (yes/partial/no) yes

4.9. This paper formally describes evaluation metrics used and explains the motivation for choosing these metrics (yes/partial/no) yes

4.10. This paper states the number of algorithm runs used to compute each reported result (yes/no) yes

4.11. Analysis of experiments goes beyond single-dimensional summaries of performance (e.g., average; median) to include measures of variation, confidence, or other distributional information (yes/no) yes

4.12. The significance of any improvement or decrease in performance is judged using appropriate statistical tests (e.g., Wilcoxon signed-rank) (yes/partial/no) yes

4.13. This paper lists all final (hyper-)parameters used for each model/algorithm in the paper's experiments (yes/partial/no/NA) yes